Exercise 1 — A Complete Introduction to Multilevel Queue Scheduling

CPU scheduling with multilevel queues: theory, design, implementation and measured results

Exercise 1 — Algorithm introduction
Course Advanced Operating Systems
Topic CPU scheduling and multilevel queue algorithms
Deliverables Bilingual report (DOCX + PDF), runnable Python code, diagrams
Repository github.com/kaspian021/Multilevel-Queue-Scheduling
Branch Exercise-1-Algorithm-Introduction
Page count as produced by the generator (more than 20 pages)
Status complete — every number below comes from an actual program run

Table of contents

3 Abstract, keywords and reading map
4 Introduction and problem statement
5 Goals, research questions and method
6 Fundamentals (1): CPU, process and thread
7 Fundamentals (2): the execution cycle and burst time
8 Fundamentals (3): evaluation metrics
9 Fundamentals (4): ready queue, context switch and utilisation
10 The MLQ algorithm: rationale and design goals
11 Queue structure and workload mapping
12 Assigning processes to queues
13 Intra-queue scheduling policies
14 Managing priority between queues
15 Flowchart of the algorithm
16 Pseudocode (part 1): the scheduler loop
17 Pseudocode (part 2): execution, preemption and aging
18 Worked numerical example (1): statement
19 Worked numerical example (2): step-by-step trace
20 Worked numerical example (3): Gantt chart and results
21 Worked numerical example (4): computation table
22 Second scenario: comparing intra-queue policies
23 Starvation analysis
24 Starvation avoidance
25 Complexity analysis
26 Python implementation (1): layout and commands
27 Python implementation (2): input, output and error handling
28 Scenarios and experimental results
29 Analysis of the results
30 Scheduling overhead
31 Advantages, disadvantages and comparison with single-queue policies
32 Limitations, assumptions and real-world applications
33 Conclusion and recommendations
34 References
35 Appendix: output inventory and requirement mapping

Abstract, keywords and reading map

Abstract

CPU scheduling is one of the fundamental duties of an operating system and it directly determines responsiveness, throughput and the fairness of resource allocation. This report presents a complete introduction to Multilevel Queue (MLQ) scheduling: a scheme that partitions the ready set into several queues, gives every queue its own scheduling policy, and uses an inter-queue policy (fixed priority or time slice) to decide which queue may use the CPU at a given instant.

Beyond the theoretical exposition, a discrete, runnable simulator was written in Python. It supports four intra-queue policies (FCFS, RR, SJF, SRTF), two inter-queue policies (fixed priority and time slice) and an optional aging extension. Four experimental scenarios were designed and executed; every waiting time, turnaround time, response time, utilisation figure and context-switch count quoted in this report was extracted from an actual run and is reproducible with a single command. The results show that MLQ gives precise control over prioritisation, but that without aging the lower queues starve: in the starvation scenario a low-priority process waited 20 ticks, which aging reduced to 6.

Keywords

CPU scheduling; multilevel queue; fixed priority; time slice; waiting time; turnaround time; response time; context switch; starvation; aging; Gantt chart; Python.

Reading map

- Sections 1–3: fundamentals, terminology and evaluation metrics.
- Sections 4–7: the MLQ algorithm, queue structure, intra- and inter-queue policies.
- Sections 8–10: flowchart, pseudocode and a fully worked numerical example.
- Sections 11–14: experiments, result analysis, starvation and aging.
- Sections 15–19: complexity, implementation, trade-offs, limits and applications.

Introduction and problem statement

Problem statement

In a time-sharing system the number of ready processes normally exceeds the number of cores. At every instant the operating system must decide which ready process gets the CPU. That decision is taken by the short-term scheduler, also called the CPU scheduler, and it is carried out by the dispatcher.

The central difficulty is that processes are not homogeneous. Some are interactive and latency-sensitive, some are long-running and CPU-bound, and some belong to the system itself. A single policy applied to all of them cannot simultaneously deliver good response time, fairness and throughput.

Why MLQ?

- FCFS is simple but yields a high average waiting time and suits interactive work poorly.
- SJF minimises average waiting time but needs the burst length in advance and can starve long jobs.
- RR gives good responsiveness but raises the context-switch overhead and the average turnaround time.
- Priority scheduling is flexible but needs a priority definition and starvation handling.
- MLQ combines these policies in layers so that each class of work can be treated differently.

Formal statement

Given n processes, each with an arrival time, a burst length and a queue level, find an execution order that respects the priority structure while keeping the evaluation metrics inside an acceptable range. In this model, time is discrete and measured in ticks.

> **Scope of the model:** The simulator is an educational, measurable model. It is not a replacement for a kernel scheduler and it does not measure real wall-clock CPU time.

Goals, research questions and method

Overall goal

To give a complete, precise and verifiable introduction to the MLQ algorithm: theory, design, pseudocode, a runnable implementation and a quantitative evaluation based on purpose-built scenarios.

Specific objectives

- Define the fundamentals: CPU, process, thread, burst and context switch.
- Give operational and mathematical definitions of waiting, turnaround and response time.
- Explain the queue structure, process-to-queue assignment and intra-queue policies.
- Compare the two inter-queue policies: fixed priority and time slice.
- Provide a complete flowchart and pseudocode that maps directly onto the code.
- Solve a full numerical example with a Gantt chart and a computation table.
- Implement the simulator in Python with error handling, automated tests and inspectable output.
- Analyse starvation and quantify the effect of the aging extension.

Research questions

- Can a multilevel structure improve interactive response time without hurting utilisation?
- Which intra-queue policy minimises average waiting time for a given workload?
- How severe is starvation in the lower queues, and how much does aging mitigate it?
- How large is the scheduling overhead when measured in context switches?

Method

- Literature-based consolidation of the theory and the terminology.
- Design of a discrete model and its implementation in Python.
- Design and automated execution of four scenarios.
- Generation of every figure from real program output, followed by numerical analysis.
- Cross-validation of the code against a hand-worked reference example.

Fundamentals (1): CPU, process and thread

The CPU and the scheduler

The CPU is a resource that can be held by exactly one process at a time. The short-term scheduler selects the next process from the ready set; the dispatcher performs the actual handover.

Process

- A process is a program in execution: address space, registers, open files and scheduling metadata.
- It moves through the states New, Ready, Running, Waiting and Terminated.
- The process control block (PCB) stores everything needed to stop and resume execution.
- The scheduler only decides between processes that are in the Ready state.

Thread

- A thread is the smallest unit of execution and belongs to a process.
- Threads of one process share memory and resources but have independent execution contexts.
- Modern operating systems usually schedule threads; this report uses the word process for simplicity.
- In the model of this exercise each input row is one independently schedulable unit.

Property	Process	Thread
Address space	private	shared with sibling threads
Creation cost	high	low
Context-switch cost	high	lower
Fault isolation	strong	weak
Unit of scheduling in modern systems	usually no	usually yes

Table 1. Processes versus threads from a scheduling point of view

Fundamentals (2): the execution cycle and burst time

CPU and I/O cycle

Execution alternates between a phase of CPU use (a CPU burst) and a phase of waiting for input/output (an I/O burst). A process starts with a CPU burst and, when it needs I/O, leaves the ready set until the I/O completes.

Burst time

Burst time is the amount of CPU time a process needs. In this exercise it is an integer number of ticks and it is assumed to be known at arrival time.

Modelling notes

- On a real system the burst length is not known in advance and must be predicted.
- Practical SJF is therefore approximate and typically uses exponential averaging.
- Knowing the burst is an explicit assumption here; it is repeated in the limitations section.
- Real workloads have heavy-tailed burst distributions: a few processes are extremely long.

Scheduler type	Decision moment	Example
Long-term	admission to the system	degree of multiprogramming
Medium-term	suspend and resume	swapping
Short-term	every tick or event	FCFS, RR, SJF, MLQ

Table 2. Levels of scheduling decision

- In non-preemptive scheduling the CPU is not released until the burst ends or the process blocks.
- In preemptive scheduling a running process can be stopped and replaced.
- The MLQ model of this exercise is preemptive between queues; inside a queue it depends on the policy.
- RR and SRTF are preemptive; FCFS and SJF are non-preemptive inside their queue.

Fundamentals (3): evaluation metrics

Symbol	Definition
AT (arrival time)	instant at which the process joins the ready set
BT (burst time)	CPU time required by the process
CT (completion time)	instant at which the process finishes
TAT (turnaround time)	CT - AT; total time spent in the system
WT (waiting time)	TAT - BT; time spent in the ready queue
RT (response time)	first start - AT
Context switch	change of the process occupying the CPU

Table 3. Notation used throughout this report

```
TAT_i = CT_i - AT_i
WT_i  = TAT_i - BT_i
RT_i  = ST_i - AT_i
AVG(WT) = (1/n) * sum(WT_i)
```

Basic metric relations

Interpretation

- Low waiting time means little of a process lifetime is lost in queues.
- Turnaround matters for batch work; response time matters for interactive work.
- CPU utilisation is busy time over total time; it is 100% here because the scheduler is work-conserving.
- The context-switch count is a proxy for scheduling overhead and must be read together with the other metrics.

Fundamentals (4): ready queue, context switch and utilisation

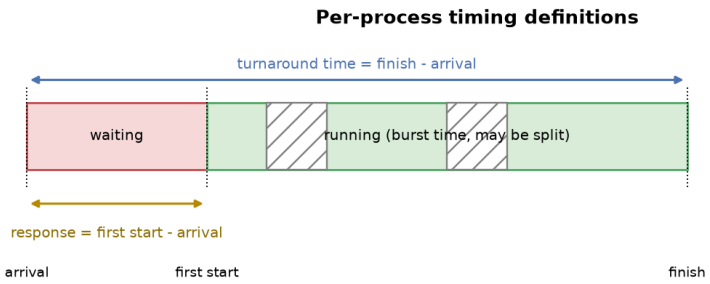


Figure 4. Visual definition of burst, waiting, turnaround and response for one process

Context switch

- A context switch saves the state of the current process and loads the state of the next one.
- It is pure overhead: no useful work is done while it happens.
- Its cost includes register save/restore, page-table updates and lost cache locality.
- In this model the switch count is derived from the execution trace so that it always agrees with the Gantt chart.

```
CPU Utilization = Busy Time / Total Time
Throughput = n / Total Time
```

CPU utilisation and throughput

- In all scenarios of this exercise utilisation is 100% because a ready process always exists.
- 100% utilisation is not a quality signal here; it only means the scheduler is work-conserving.
- The policies differ in waiting time, turnaround time and response time, not in utilisation.

The MLQ algorithm: rationale and design goals

Definition

Multilevel queue scheduling partitions the ready set into several separate queues. Each queue has its own priority level and its own scheduling policy. Processes are assigned to a queue based on properties such as their class, importance or memory requirements, and in the classical model they never move between queues afterwards.

Design goals

- Policy separation: every class of work receives the policy that suits it best.
- Low overhead: there is no need to keep all processes in a single ordered structure.
- Priority control: critical work can be guaranteed to run first.
- Analysability: the behaviour of each level can be studied independently.
- Stability: changing the policy of one queue does not disturb the others.

Property	MLQ	MLFQ
Number of queues	fixed	fixed
Movement between queues	no (classical model)	yes, driven by observed behaviour
Extra parameters	few	many
Implementation complexity	low	high

Table 5. Plain multilevel queue versus multilevel feedback queue

Approach taken: Aging is presented here only as an optional starvation-avoidance extension. Its setting is stated explicitly next to every table of results.

Queue structure and workload mapping

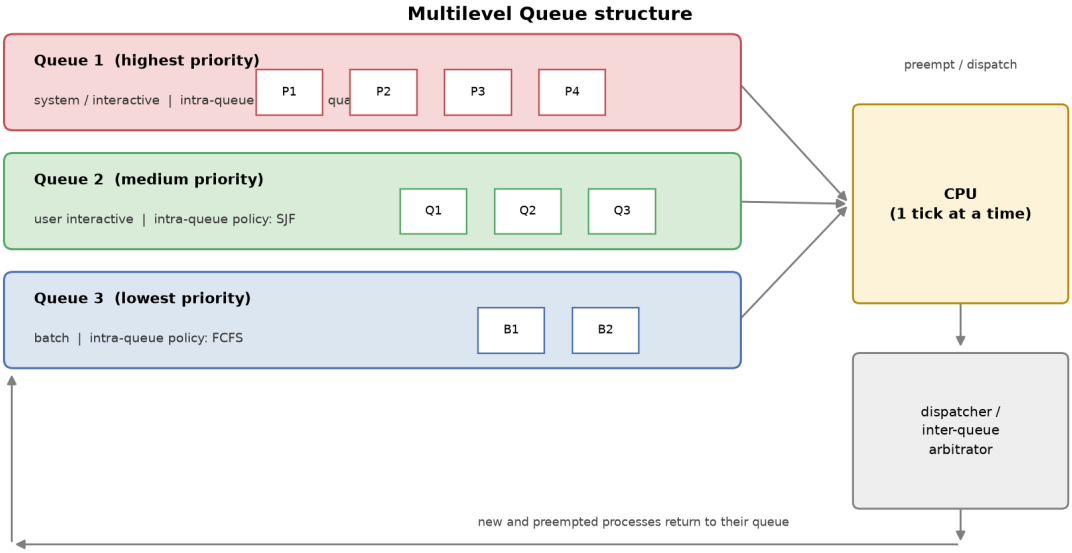


Figure 6. Overall structure: three queues, intra-queue policies and the inter-queue arbitrator

Components

- Queue set: three queues numbered 1 to 3 in this project; a lower number means a higher priority.
- Intra-queue policy: FCFS, RR, SJF or SRTF for each queue.
- Inter-queue policy: fixed priority or time slice, drawn as the arbitrator in the figure.
- Execution unit: one CPU that is granted to one process per tick.

Queue	Workload class	Suggested policy	Reason
1	system and critical interactive	RR with a short quantum	fast response
2	normal interactive	SJF or SRTF	lower average waiting time
3	batch and background	FCFS	simplicity and minimal overhead

Table 7. A practical mapping from workload classes to queues

Assigning processes to queues

Assignment criteria

- Process type: system, interactive, batch or real-time.
- Memory requirement: large jobs may be placed in a queue with a different policy.
- Administrative priority: a priority set by policy or by the administrator.
- Measured behaviour: the I/O intensity observed during earlier runs.
- Time sensitivity: jobs with deadlines are placed in a higher queue.

Assignment rule used here

In the JSON input every process carries an explicit queue number. This is the most transparent option: the scenario designer performs the assignment and the algorithm only validates that the number refers to a defined queue.

Error	Program message	Behaviour
Unknown queue number	queue 7 not in [1]	exit code 2
Missing field	missing field 'burst'	exit code 2
Zero or negative burst	burst must be > 0	exit code 2
Negative arrival	arrival must be >= 0	exit code 2

Table 8. Real program behaviour for invalid input

Design note: Static assignment is the main difference between MLQ and MLFQ. If movement driven by observed behaviour is required, use MLFQ or the aging extension.

Intra-queue scheduling policies

FCFS — first come, first served

- Non-preemptive and ordered by arrival.
- Implemented with a plain FIFO; selection costs O(1).
- Convoy effect: one long process delays everything behind it.
- Well suited to batch queues where low overhead matters.

RR — round robin

- Preemptive with a fixed quantum; after its slice a process goes to the tail of its queue.
- Gives good response time for interactive work.
- A very small quantum inflates the context-switch overhead.
- A very large quantum degenerates towards FCFS.

SJF — shortest job first

- Non-preemptive and theoretically optimal for average waiting time.
- Requires knowing or estimating the burst length.
- Can starve long jobs if short jobs keep arriving.

SRTF — shortest remaining time first

- The preemptive form of SJF; a newly arrived shorter job interrupts the running one.
- Reduces average waiting time further compared with SJF.
- Costs more overhead because the remaining time must be re-evaluated continuously.

Managing priority between queues

Option 1: fixed priority

The highest non-empty queue always wins. If a process becomes ready in a higher queue, the process running from a lower queue is preempted immediately. The method is simple, deterministic and predictable, but a continuous stream of high priority work starves the lower queues.

Option 2: time slice

Each queue receives a time budget; when the budget is exhausted service moves to the next queue. In theory this removes starvation and balances response time, but it increases the number of context switches.

Inter-queue policy	Avg waiting	Avg turnaround	Avg response	Context switches
fixed priority	8.40	13.20	7.80	5
time slice = 2	14.20	19.00	3.60	12
time slice = 3	12.80	17.60	6.00	8
time slice = 5	13.20	18.00	8.60	6

Table 9. Measured results on timeslice.json with four configurations

- A slice of 2 cut the response time from 7.80 to 3.60, an improvement of about 54%.
- The same change raised the average waiting time from 8.40 to 14.20.
- Context switches went from 5 to 12, that is, more overhead.
- Choosing between the two options is an explicit trade-off between responsiveness and overhead.

Flowchart of the algorithm

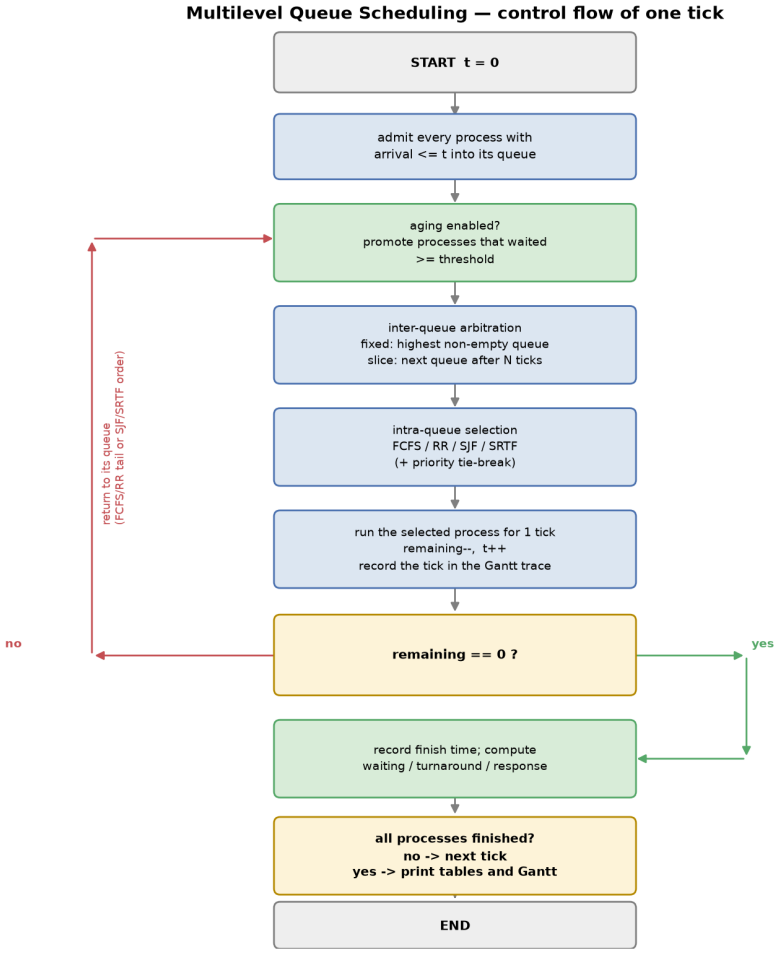


Figure 10. Complete flowchart of one tick of the multilevel queue scheduler

How to read it

- Step 1: admit every process whose arrival time has been reached.
- Step 2: if aging is enabled, promote processes that have waited long enough.
- Step 3: the inter-queue arbitrator decides which queue may use the CPU.
- Step 4: the intra-queue policy selects the process inside that queue.
- Step 5: execute one tick, decrement the remaining time and record the trace.
- Step 6: if the process finished, record its completion time; otherwise return it to its queue.

Pseudocode (part 1): the scheduler loop

```
MLQ_SCHEDULER(processes, queue_config, aging, inter_policy, inter_slice):
    levels  <- sorted(queue_config.keys())        # 1 = highest priority
    queues  <- { q: empty FIFO list  for q in levels }
    pending <- sort(processes by (arrival, pid))
    t <- 0 ;  current <- NULL ;  slice_used <- 0 ;  budget <- inter_slice
    while unfinished processes remain:
        ADMIT(t)                                  # arrival <= t joins its queue
        if aging > 0:  APPLY_AGING(t, aging)      # promote long waiters
        top <- the lowest-numbered non-empty queue, or NULL

        # ---- inter-queue arbitration ----
        if inter_policy == FIXED:
            if current != NULL and top < current.queue:
                push_front(queues[current.queue], current) ; current <- NULL
        else:  # TIME SLICE
            if current != NULL and budget <= 0:
                push_back(queues[current.queue], current)  ; current <- NULL
                budget <- inter_slice

        # ---- intra-queue selection ----
        if current == NULL and top != NULL:
            q    <- SELECT_QUEUE(top, levels, inter_policy)
            current <- SELECT_PROCESS(queues[q], queue_config[q].policy)
            remove current from queues[q]
            if current.first_start == NULL: current.first_start <- t
            slice_used <- 0
```

Pseudocode part 1: admission, inter-queue arbitration and process selection

- SELECT_QUEUE returns the highest non-empty queue in fixed-priority mode.
- SELECT_QUEUE returns the next queue of a cyclic order in time-slice mode.
- SELECT_PROCESS applies FCFS, RR, SJF or SRTF according to the queue policy.

Pseudocode (part 2): execution, preemption and aging

```
        # ---- SRTF intra-queue preemption ----
        if current != NULL and queue_config[current.queue].policy == SRTF:
            best <- the ready process in current.queue with min remaining
            if best.remaining < current.remaining:  swap(current, best)

        if current == NULL:
            trace.append(IDLE) ; t <- t + 1 ; continue

        trace.append(current.pid)
        current.remaining <- current.remaining - 1
        slice_used <- slice_used + 1 ;  budget <- budget - 1 ;  t <- t + 1

        if current.remaining == 0:
            current.finish <- t ;  current <- NULL ;  budget <- 0
        else if (policy == RR and slice_used >= quantum)
                or (inter_policy == SLICE and budget <= 0):
            ADMIT(t)                              # new arrivals go first
            push_back(queues[current.queue], current) ; current <- NULL

    # ---- metrics ----
    for p in processes:
        p.turnaround = p.finish - p.arrival
        p.waiting    = p.turnaround - p.burst
        p.response   = p.first_start - p.arrival
    return trace, metrics, per_queue_statistics

APPLY_AGING(t, threshold):
    for each queue q from the second-highest downwards:
        for p in queues[q]:
            if t - p.wait_since >= threshold:
                move p one level up ; p.wait_since <- t ; promotions <- promotions + 1
```

Pseudocode part 2: one tick of execution, preemption conditions and aging

- Each tick consumes exactly one unit of the selected process burst.
- Preemption happens at three points: a higher queue, quantum expiry and SRTF.
- Aging promotes one level at a time so that the behaviour stays controllable.

Worked numerical example (1): statement

Three queues are configured as follows: queue 1 uses RR with a quantum of 2, queue 2 uses SJF and queue 3 uses FCFS. Five processes arrive at different instants. The task is to determine the execution order, the Gantt chart and the scheduling metrics.

Process	Arrival	Burst	Queue	Queue policy
P1	0	5	3	FCFS
P2	1	4	1	RR (q=2)
P3	2	3	2	SJF
P4	3	2	1	RR (q=2)
P5	4	1	2	SJF

Table 11. Input data of the first worked example (baseline.json)

Execution rules for this example

- Inter-queue arbitration is fixed priority: queue 1 beats queue 2, queue 2 beats queue 3.
- Preemption is allowed: a new arrival in a higher queue stops the running process.
- In queue 1 a process returns to the tail after two ticks.
- In queue 2 the shortest burst is selected and runs to completion.
- Aging is disabled so that the pure algorithm behaviour is visible.

Worked numerical example (2): step-by-step trace

Interval	Decision	Explanation
0-1	P1	the only ready process; it runs from queue 3
1-3	P2	arrival in queue 1 preempts P1; two RR ticks
3-5	P4	P4 has just arrived in queue 1 and runs to completion
5-7	P2	the remaining two ticks of P2
7-8	P5	queue 2 with SJF picks the shortest job
8-11	P3	the only process left in queue 2
11-15	P1	back to queue 3 for its four remaining ticks

Table 12. Step-by-step trace of the first worked example

- At t=1 P2 reaches queue 1 and, because priority is fixed, P1 from queue 3 is stopped.
- At t=3 the RR slice of P2 is exhausted and the newly arrived P4 moves ahead in queue 1.
- At t=7 queue 2 holds P3 (burst 3) and P5 (burst 1); SJF selects P5.
- At t=11 queues 1 and 2 are empty and P1 finishes its four remaining ticks.

Cross-check with program output: This trace is exactly the one produced by the program and stored in output/scenario_baseline.txt.

Worked numerical example (3): Gantt chart and results

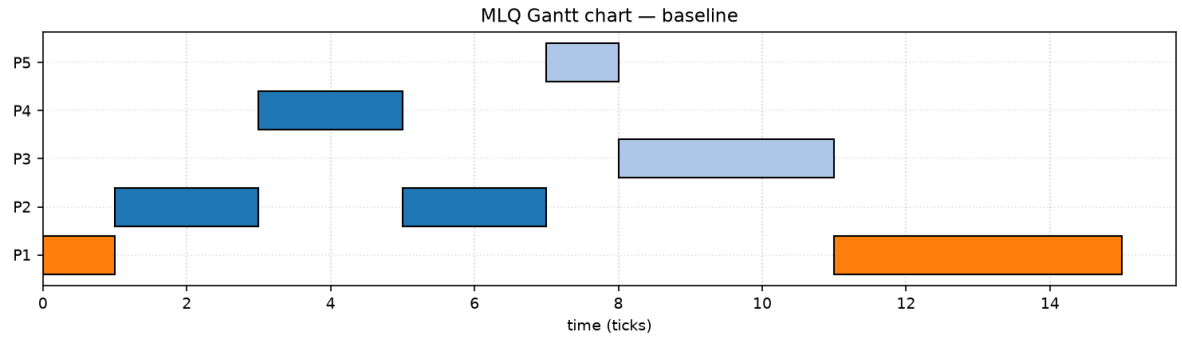


Figure 13. Gantt chart of the first worked example, generated from a real run

- The sum of the non-idle intervals is 15 and the sum of the bursts is 5+4+3+2+1 = 15.
- There is no idle interval, so CPU utilisation is 100%.
- There are 7 intervals, matching the 7 CPU allocations.
- Process-to-process context switches: 6; preemptions: 2.

Worked numerical example (4): computation table

Process	Queue	Arrival	Burst	Finish	Waiting	Turnaround	Response
P1	3	0	5	15	10	15	0
P2	1	1	4	7	2	6	0
P3	2	2	3	11	6	9	6
P4	1	3	2	5	0	2	0
P5	2	4	1	8	3	4	3

Table 14. Complete results of the first worked example

- P1: turnaround = 15 - 0 = 15; waiting = 15 - 5 = 10; response = 0 - 0 = 0.
- P2: turnaround = 7 - 1 = 6; waiting = 6 - 4 = 2; response = 1 - 1 = 0.
- P3: turnaround = 11 - 2 = 9; waiting = 9 - 3 = 6; response = 8 - 2 = 6.
- P4: turnaround = 5 - 3 = 2; waiting = 2 - 2 = 0; response = 3 - 3 = 0.
- P5: turnaround = 8 - 4 = 4; waiting = 4 - 1 = 3; response = 7 - 4 = 3.
- Average waiting = (10+2+6+0+3)/5 = 4.20; average turnaround = 36/5 = 7.20.

Queue	Count	Avg waiting	Avg turnaround	Max turnaround
1	2	1.00	4.00	6
2	2	4.50	6.50	9
3	1	10.00	15.00	15

Table 15. Per-queue statistics for the first worked example

Second scenario: comparing intra-queue policies

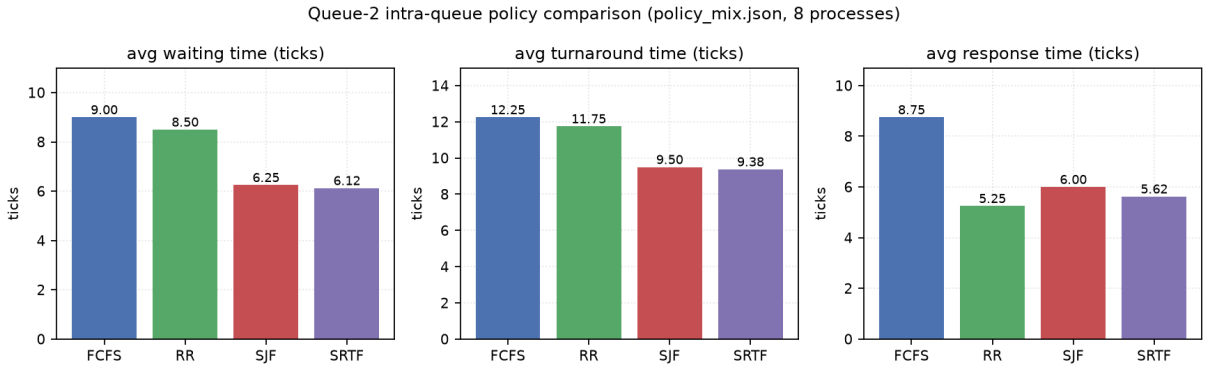


Figure 16. Four intra-queue policies on one identical workload (policy_mix.json)

Queue-2 policy	Avg waiting	Avg turnaround	Avg response	Context switches
FCFS	9.00	12.25	8.75	8
RR (q=2)	8.50	11.75	5.25	13
SJF	6.25	9.50	6.00	8
SRTF	6.12	9.38	5.62	9

Table 17. Measured comparison of the intra-queue policies

- SRTF gives the lowest average waiting time (6.12) and the lowest average turnaround (9.38).
- RR improves the response time from 8.75 to 5.25 over FCFS, but raises context switches from 8 to 13.
- SJF and SRTF differ only slightly on this workload because preemption rarely finds a better candidate.
- These results belong to this workload and must not be generalised without further experiments.

Starvation analysis

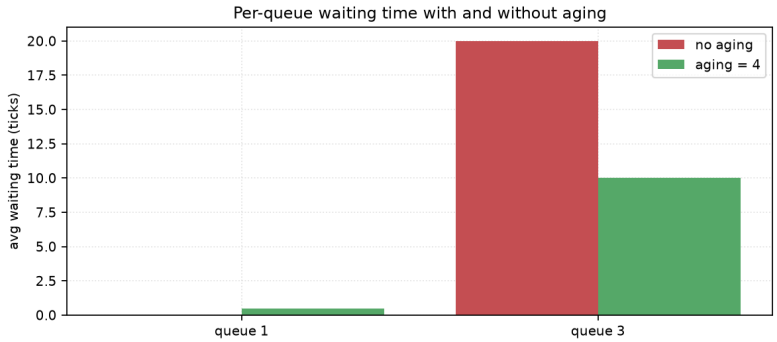


Figure 18. Per-queue waiting time in the starvation scenario, with and without aging

Starvation is the situation in which a ready process waits for an unbounded time because the resource is always given to somebody else. Under MLQ with fixed priority, any continuous stream of work in the upper queues prevents the lower queues from ever running.

Metric	Queue 1	Queue 3
Process count	10	1
Average waiting time	0.00	20.00
Average turnaround time	2.00	21.00
Response time	0.00	20.00

Table 19. starvation.json without aging: a complete gap between the top and bottom queue

- Process L1 in queue 3 has a burst of 1 but has to wait 20 ticks.
- Waiting time grows linearly with the length of the high-priority stream: 2 ticks per extra process.
- This behaviour is asserted by automated tests for stream lengths 2, 5 and 10.
- Utilisation is still 100% here, which shows that utilisation does not reveal starvation.

Starvation avoidance

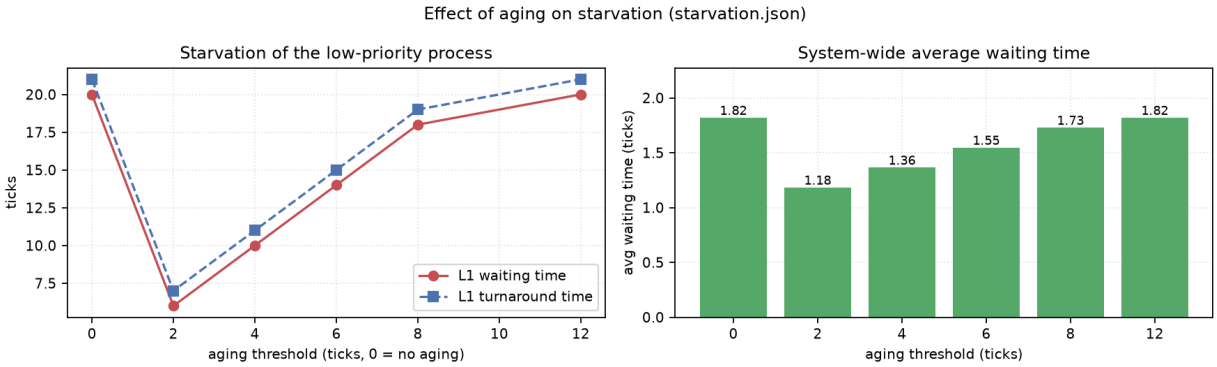


Figure 20. Effect of the aging threshold on the low-priority waiting time and on the system average

With aging, any process that has waited longer than a threshold in the ready state is promoted one level. Repeating this eventually moves the process to the top queue, where it is guaranteed to run. Aging effectively turns the priority into a function of the elapsed waiting time.

Aging threshold	L1 waiting	L1 turnaround	System avg waiting	Promotions
0 (disabled)	20	21	1.82	0
2	6	7	1.18	2
4	10	11	1.36	2
6	14	15	1.55	2
8	18	19	1.73	2
12	20	21	1.82	1

Table 21. Aging threshold sweep on starvation.json (real output)

- With a threshold of 2 the waiting time of L1 drops from 20 to 6, an improvement of 70%.
- At the same time the system-wide average waiting time falls from 1.82 to 1.18 rather than rising.
- A larger threshold promotes later, so the effect of aging weakens.
- Aging changes the behaviour of classical MLQ and must therefore always be reported explicitly.

- Time slicing between queues: guarantees that every queue receives a share of the CPU.
- Priority quotas: cap the percentage of CPU time each level may consume.
- Manual promotion: the administrator moves a process in exceptional cases.
- MLFQ: automatic movement driven by the observed behaviour of the process.

Complexity analysis

Let n be the number of processes, Q the number of queues and T the length of the simulation horizon in ticks. T equals the sum of all bursts plus all idle time.

Operation	Cost	Explanation
Admitting processes	O(n log n)	initial sort by arrival time
Selecting the queue	O(Q)	linear scan for the highest non-empty queue
Selection under FCFS/RR	O(1)	take from the head of the queue
Selection under SJF/SRTF	O(m)	minimum over the m ready processes of that queue
One tick of execution	O(1) to O(m)	depending on the queue policy
Whole simulation	O(T * (Q + m))	ticks times the cost per tick

Table 22. Time complexity of the simulator components

- Space for processes and queues: O(n).
- Queue configuration map: O(Q).
- Execution trace for the Gantt chart: O(T) in the worst case.
- In this implementation the trace is stored in a compressed form and is usually far smaller than T.
- A production scheduler would use a heap or a bitmap to reach O(log m) or O(1) selection.

Python implementation (1): layout and commands

```
exercise-1/
  README.md              branch guide and file overview
  REPORT_FA.md           this report in Persian
  REPORT_EN.md           this report in Markdown
  requirements.txt       Python dependencies
  code/
    mlq.py               simulator and command-line interface
    run_experiments.py   runs every scenario and rebuilds the charts
    make_diagrams.py     draws the structure, flowchart and concept figures
    scenarios/*.json     valid and invalid scenarios
  tests/test_mlq.py      core tests
  tests/test_mlq_extra.py tests for the newer features
  diagrams/              generated figures
  output/                text, CSV and JSON output
  docs/                  DOCX and PDF versions of the report
```

Directory layout of exercise 1

```
cd exercise-1
python code/mlq.py code/scenarios/baseline.json
python code/mlq.py code/scenarios/policy_mix.json --policy 2=SRTF --quiet
python code/mlq.py code/scenarios/baseline.json --png g.png --csv r.csv --json r.json
python code/mlq.py --interactive
python code/run_experiments.py
python code/make_diagrams.py
python -m pytest tests/ -q
```

Commands to reproduce every result in this report

Python implementation (2): input, output and error handling

```
{
  "inter_policy": "fixed",        # fixed | slice
  "inter_slice": 2,
  "aging": 0,
  "queues": {
    "1": {"policy": "RR", "quantum": 2},
    "2": {"policy": "SJF"},
    "3": {"policy": "FCFS"}
  },
  "processes": [
    {"pid": "P1", "arrival": 0, "burst": 5, "queue": 3},
    {"pid": "P2", "arrival": 1, "burst": 4, "queue": 1, "priority": 0}
  ]
}
```

Scenario file format (baseline.json)

Error handling

- All input problems raise InputError and terminate with exit code 2.
- Checks cover the empty list, duplicate identifiers, negative arrival and non-positive burst.
- An unknown policy or an invalid quantum is detected before the simulation starts.
- Invalid scenario files are kept in the scenarios folder so the error behaviour is reproducible.
- A max_ticks guard prevents an infinite loop in case of a modelling mistake.

Test coverage

- 59 automated pytest tests were executed and all of them pass.
- They cover Gantt correctness, metric identities, policies, starvation, aging and input errors.
- Newer tests compare the context-switch count directly against the Gantt chart.
- The full test output is stored in output/pytest_result.txt.

Scenarios and experimental results

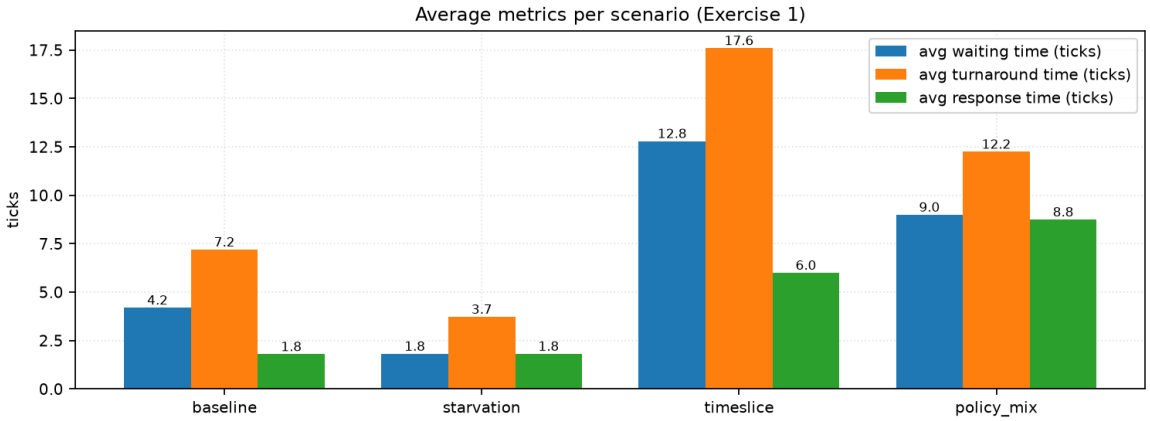


Figure 23. Average metrics across the four experimental scenarios

Scenario	n	Avg waiting	Avg turnaround	Avg response	Utilisation	Ctx switches
baseline	5	4.20	7.20	1.80	100%	6
starvation	11	1.82	3.73	1.82	100%	10
timeslice	5	12.80	17.60	6.00	100%	8
policy_mix	8	9.00	12.25	8.75	100%	8

Table 24. Summary of every scenario (from output/experiments.json)

- baseline: three queues with three different policies and five processes; the teaching reference.
- starvation: ten processes in the top queue and one in the bottom queue; used to measure starvation.
- timeslice: five long-burst processes, used to compare fixed priority with time slicing.
- policy_mix: eight processes, five of them in queue 2, used to compare intra-queue policies.

Analysis of the results

- Utilisation is 100% everywhere because the scheduler is work-conserving and no idle time appears.
- The highest average waiting time occurs in the timeslice scenario (12.80), a direct consequence of sharing time between queues.
- The lowest average waiting time occurs in the starvation scenario (1.82), but that figure is misleading because one process waited 20 ticks.
- This contrast shows that averages alone are not sufficient; dispersion must be reported as well.

Design decision	Benefit	Cost
fixed priority	excellent response for the top queue	starvation of the lower queues
time slice	balanced response across queues	more switches and more waiting
RR with a short quantum	better responsiveness	higher overhead
SJF/SRTF	less waiting	needs the burst and risks starvation
aging	removes starvation	deviates from classical MLQ

Table 25. The main trade-offs in multilevel queue design

- The first worked example was computed by hand and matched the program output exactly.
- Automated tests assert the identity waiting = turnaround - burst for every scenario.
- The context-switch count is derived from the trace, not counted separately, so it always agrees with the Gantt chart.

Scheduling overhead

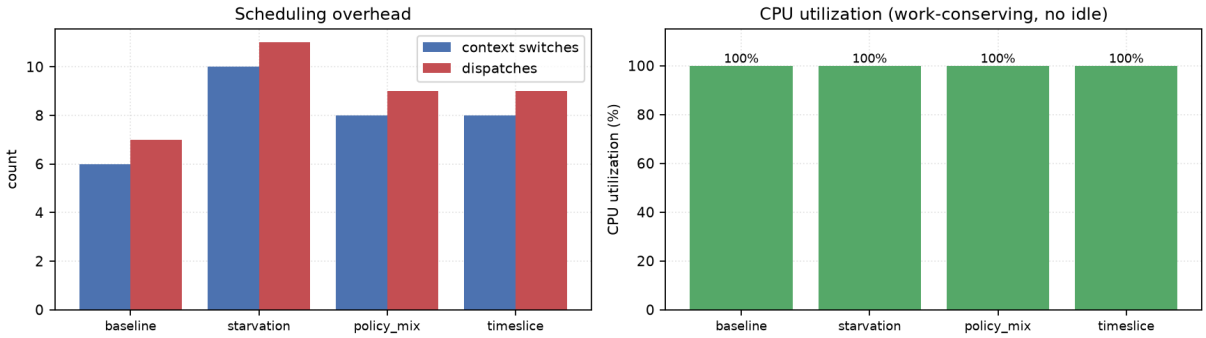


Figure 26. Context switches, dispatches and CPU utilisation per scenario

- The starvation scenario has 11 processes and 10 context switches, roughly one switch per process.
- The baseline scenario has 5 processes, 7 dispatches and 6 switches, caused by preemption in queue 1.
- Time slicing with a budget of 2 produced 12 switches, a 140% increase over fixed priority.
- Utilisation is unaffected by switching in this model because a switch is assumed to cost nothing.

Measurement limit: On a real system every switch costs microseconds and invalidates cache contents, so this chart should be read as a lower bound on the overhead.

Advantages, disadvantages and comparison with single-queue policies

Advantages

- Policy separation: each class of work can be optimised independently.
- Explicit priority control and guaranteed responsiveness for critical work.
- Low overhead in the queues that use FCFS.
- Simpler to implement than feedback-based algorithms.
- Can be combined with advanced policies inside each level.

Disadvantages

- Starvation of the lower queues when the upper queues are busy.
- Inflexibility: in the classical model a process never moves between queues.
- Manual tuning of the number of queues, the quantum and the policy of each queue.
- Results depend heavily on the quality of the initial process-to-queue assignment.

Criterion	FCFS	RR	SJF	MLQ
Interactive response	poor	good	medium	good (top queue)
Average waiting time	high	medium	optimal	depends on assignment
Overhead	low	medium	low	low to medium
Starvation risk	none	none	yes	yes (lower queues)
Flexibility	low	low	low	high

Table 27. MLQ compared with single-queue policies

Limitations, assumptions and real-world applications

Model assumptions

- Time is discrete and measured in ticks.
- The burst length is known in advance; on a real system it must be predicted.
- The context-switch cost is zero and does not influence the metrics.
- There is a single processor; the multiprocessor case is not studied.
- Processes are independent: no dependencies and no synchronisation.
- No I/O operation occurs in the middle of a burst.

Experimental limitations

- The scenarios are synthetic and small so that hand calculation stays possible.
- The results belong to these scenarios and do not generalise automatically.
- The simulator runs in user space and does not measure a real kernel scheduler.
- Energy, cache effects and memory latency were not measured.

Real-world applications

- Separating system work from user work in time-sharing operating systems.
- Prioritising banking or payment transactions over reporting jobs.
- Managing print queues and batch job queues.
- Packet scheduling in networks with several service classes.
- Resource allocation in databases between interactive queries and batch jobs.

Conclusion and recommendations

Multilevel queue scheduling is a structured answer to workload heterogeneity. This exercise showed that by separating the queues and giving each level its own policy, the algorithm can substantially improve the responsiveness of interactive work: in the timeslice scenario the response time fell from 7.80 to 3.60 units. At the same time the average waiting time rose from 8.40 to 14.20 and the number of context switches grew from 5 to 12.

It was also shown that without a protective mechanism the lower queues starve completely. Aging with a threshold of 2 cut the waiting time of the low-priority process from 20 to 6 units, while the system-wide average waiting time improved as well. In this scenario the cost of removing starvation was far lower than its benefit.

Recommendations for further work

- Add a multilevel feedback queue (MLFQ) and compare it with MLQ on one workload.
- Model the real cost of a context switch and its effect on utilisation.
- Add I/O operations and measure response time in their presence.
- Extend the model to several processors and study load balancing.
- Replace the synthetic scenarios with traced real workloads.

Key message: MLQ is a powerful but parameter-driven tool: the quality of its outcome depends above all on a correct process-to-queue assignment and on a well-tuned inter-queue policy.

References

Primary references

- Silberschatz, A., Galvin, P. B., and Gagne, G. — Operating System Concepts — chapters on CPU scheduling.
- Tanenbaum, A. S., and Bos, H. — Modern Operating Systems — scheduling sections.
- Stallings, W. — Operating Systems: Internals and Design Principles — scheduling chapter.
- Liu, C. L., and Layland, J. W. — Scheduling Algorithms for Multiprogramming in a Hard-Real-Time Environment — Journal of the ACM, 1973.

Supplementary references

- Official Python documentation for the collections and dataclasses modules.
- matplotlib documentation for figure generation.
- pytest documentation for automated testing.
- python-docx and reportlab documentation for DOCX and PDF generation.

Citation status: The textbooks above are listed as the standard references of the operating systems course. If the instructor requires exact page numbers or edition years, the copies available to the student must be checked and the citations updated.

Appendix: output inventory and requirement mapping

File	Description
output/scenario_baseline.txt	Gantt chart and full table of the reference scenario
output/scenario_starvation.txt	results of the starvation scenario
output/scenario_timeslice.txt	results of the time-slice scenario
output/scenario_policy_mix.txt	results of the policy comparison scenario
output/experiments.json	every number in machine-readable form
output/experiments_summary.txt	text summary of all experiments
output/pytest_result.txt	output of the test run
diagrams/fig_mlq_structure.png	queue structure diagram
diagrams/fig_flowchart.png	algorithm flowchart
diagrams/gantt_*.png	Gantt charts of every scenario
diagrams/chart_*.png	comparison charts

Table 28. Output inventory of exercise 1

Evaluation item	Status	Where
Theory and definitions	complete	fundamentals pages
Worked numerical example	complete	worked example pages
Flowchart	complete	flowchart page
Pseudocode	complete	pseudocode pages
Runnable code	complete	code folder and 59 tests
Comparison table	complete	several pages
Persian and English versions	complete	REPORT_FA and REPORT_EN
DOCX and PDF versions	complete	docs folder
Exercise 5 clip	not applicable here	documented in the project index

Table 29. Requirement mapping for exercise 1 (no grade is assumed)